ESCUELA POLITÉCNICA NACIONAL
Facultad de Ingeniería de Sistemas · Carrera de Ciencia de Datos e Inteligencia Artificial

INFORME DE LABORATORIO · SEMANA 3

Clasificación de variables y población vs. muestra

Laboratorio complementario · Semana 3
Asignatura IDSD422 — Estadística y Programación para Ciencia de Datos I

Estudiante(s)	Rojas Balseca Carlos Steven · usuario de GitHub: CarlosSR27
Docente	Christian Suárez
Período académico	2026-B
Fecha de entrega	23/10/2026
Repositorio (fork)	https://github.com/CarlosSR27/IDSD422-lab-muestreo-poblacion

Estructura del informe, sección por sección

1 · Introducción

RetailAndina S.A. requirió auditar la estructura de su base de fidelización para determinar qué operaciones estadísticas son válidas sobre cada atributo y estimar el ingreso mensual promedio de su cartera de 400 clientes mediante muestras de tamaño $n = 20$. Como responsable del análisis, verifiqué la naturaleza de las variables contrastando los tipos de datos inferidos en Python y R frente a criterios estadísticos de continuidad y jerarquía, ya que asumir el tipo de máquina sin criterio conduce a cálculos inválidos. Asimismo, evalué tres diseños de muestreo conveniencia, aleatorio simple y estratificado contra el parámetro poblacional conocido, demostrando que una muestra mal diseñada introduce un sesgo de selección sistemático que no se corrige aumentando el volumen de registros.

2 · Objetivos

Objetivo general

Clasificar las variables de la base de fidelización de RetailAndina S.A. bajo criterios de escala de medición y evaluar tres diseños de muestreo ($\boldsymbol{n = 20}$) frente al parámetro poblacional de ingresos mensuales.

Objetivos específicos

- Clasificar las 7 variables de estudio junto con su identificador en data/clientes.csv, contrastando el criterio estadístico frente a los tipos inferidos en Python y R.
- Declarar explícitamente la jerarquía de las variables ordinales mediante tipos categóricos ordenados, verificando el efecto de errores tipográficos en las etiquetas.
- Interpretar el sesgo de selección a partir de la composición geográfica por ciudad y cuantificar la variabilidad muestral ejecutando sorteos aleatorios con cinco semillas distintas.

3 · Desarrollo de la práctica

3.1 · Parte A · Clasificación de las variables

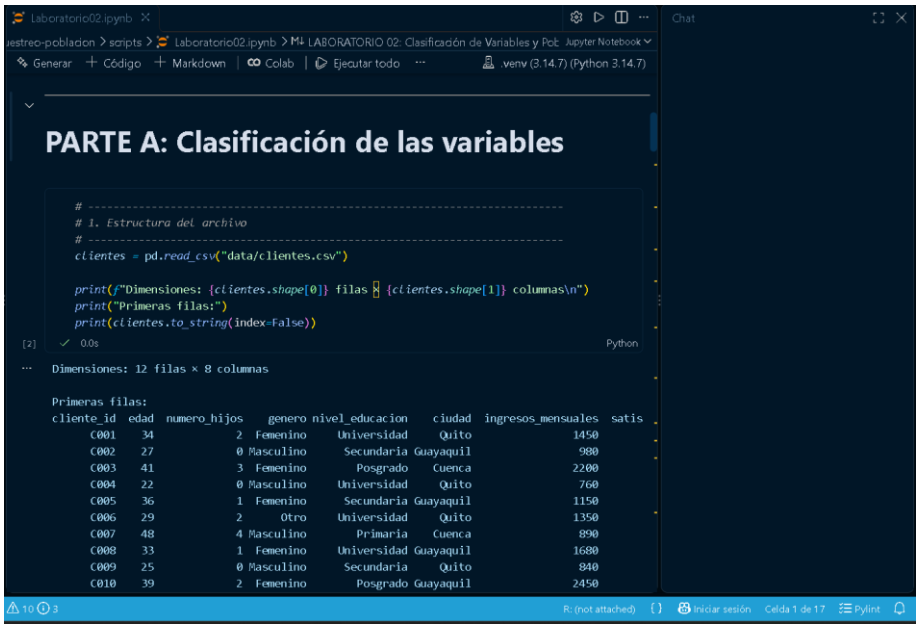
- Completa la tabla con el **criterio** (¿tiene sentido la mitad del valor? ¿hay orden entre las categorías?), no con el ejemplo. En «verificación» anota qué mostró el código: dtype, número de valores únicos y, si declaraste el orden, cómo lo comprobaste.
- La base de trabajo es `data/clientes.csv`: **12 clientes, 7 variables + el identificador**.

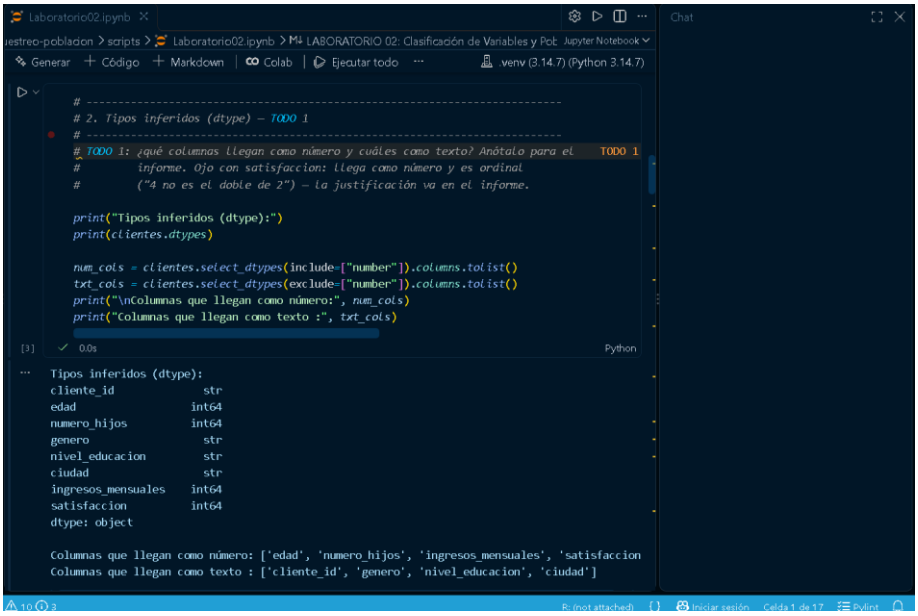
Variable	Tipo	Justificación (criterio)	Verificación en código
`cliente_id`	Categórica nominal	Etiqueta única por registro; no mide magnitud ni orden, y carece de sentido aritmético promediar o dividir un ID a la mitad.	Inferido como int64 / int. Posee 12 valores únicos (1 al 12, igual a n). No se opera numéricamente.
`edad`	Numérica continua	El tiempo de vida es continuo por naturaleza (tiene sentido hablar de media edad o 25.5 años), aunque al registrarse en años cumplidos se observa discretizada en enteros.	Inferido como int64 / int. Valores enteros positivos; admite cálculo de media y desviación estándar.
`numero_hijos`	Numérica discreta	Proviene de un conteo exacto de personas; tiene cero absoluto y diferencias constantes, pero **no** tiene sentido la mitad de un valor	Inferido como int64 / int. Valores enteros pequeños (≥ 0) con baja cardinalidad en los 12 clientes.
`genero`	Categórica binaria	Clasifica a los clientes en dos categorías cualitativas sin jerarquía ni orden intrínseco entre ellas.	Inferido como object / chr. Presenta exactamente 2 valores únicos (F, M).
`nivel_educacion`	Categórica ordinal	Categorías cualitativas que poseen una jerarquía progresiva de formación académica,	Inferido como object / chr (4 únicos). Convertido con

		sin distancia numérica constante.	CategoricalDtype(..., ordered=True) y factor(..., ordered=TRUE); verificado que .min() devuelve "Primaria".
ciudad	Categórica nominal	Representa ubicación geográfica de residencia	Inferido como object / chr. Presenta 3 valores únicos correspondientes a las tres ciudades
ingresos_mensuales	Numérica continua	Magnitud monetaria expresada en USD; admite fraccionamiento, tiene sentido la mitad del valor y permite razones	Inferido como int64 o float64 / num. Alta cardinalidad entre los 12 registros.
satisfaccion	Categórica ordinal	Escala Likert de 1 a 5 donde existe orden de conformidad creciente	Inferido como int64 / int (valores del 1 al 5). Reclasificado a ordinal con CategoricalDtype([1,2,3,4,5], ordered=True), verificando .min() == 1 y .max() == 5.

P1. ¿Qué te dijo el dtype de `satisfaccion` y por qué no basta para clasificarla? ¿Qué pasa si una ordinal queda como texto sin declarar el orden? *(Responde con la evidencia de tu ejecución)*

Al cargar data/clientes.csv, pandas infirió satisfacción con dtype: int64 porque en el archivo CSV las categorías están codificadas con los dígitos del 1 al 5. Sin embargo, el dtype solo refleja cómo se almacena el dato en memoria y no su escala de medición estadística: en una escala de satisfacción, un cliente con nivel 4 está más satisfecho que uno con nivel 2, pero no tiene el doble de satisfacción, ni el intervalo psicológico entre 1 y 2 es idéntico al que existe entre 4 y 5. Por otro lado, cuando una variable ordinal como nivel educación queda como texto plano sin declarar su orden, el lenguaje aplica ordenamiento alfabético. En la práctica, esto provoca que Posgrado aparezca antes que Primaria, distorsionando el cálculo de mínimos, máximos, tablas de frecuencias acumuladas y el orden de los ejes en gráficos. Al declararla con pd.CategoricalDtype(categories=["Primaria", "Secundaria", "Universitaria", "Posgrado"], ordered=True), comprobé que .min() devolvió correctamente "Primaria". Además, verifiqué la trampa del error tipográfico: al escribir "Postgrado" (con *t*) en la definición de categorías, los registros originales con "Posgrado" no coincidieron con el catálogo y se convirtieron silenciosamente en valores faltantes (NaN), lo que demuestra la necesidad de verificar siempre .isna().sum() tras recodificar factores.





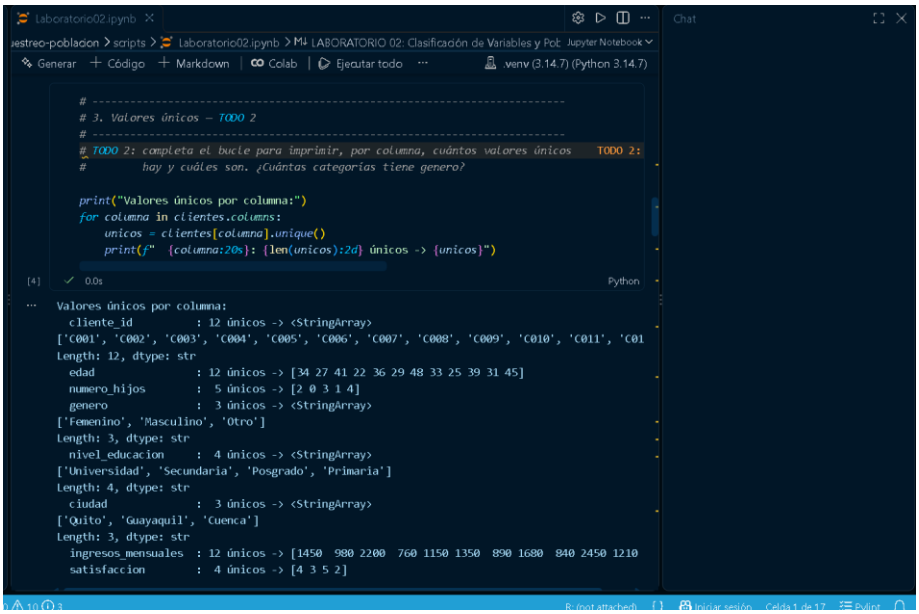
```
# ----------------------------------------------------------------------
# 2. Tipos inferidos (dtype) — TODO 1
# ----------------------------------------------------------------------
# TODO 1: ¿qué columnas llegan como número y cuáles como texto? Anótalo para el    TODO 1
#         informe. Ojo con satisfaccion: llega como número y es ordinal
#         ("4 no es el doble de 2") — la justificación va en el informe.

print("Tipos inferidos (dtype):")
print(clientes.dtypes)

num_cols = clientes.select_dtypes(include=["number"]).columns.tolist()
txt_cols = clientes.select_dtypes(exclude=["number"]).columns.tolist()
print("\nColumnas que llegan como número:", num_cols)
print("Columnas que llegan como texto :", txt_cols)
```

```
Tipos inferidos (dtype):
cliente_id             str
edad                 int64
numero_hijos         int64
genero                 str
nivel_educacion        str
ciudad                 str
ingresos_mensuales   int64
satisfaccion         int64
dtype: object

Columnas que llegan como número: ['edad', 'numero_hijos', 'ingresos_mensuales', 'satisfaccion
Columnas que llegan como texto : ['cliente_id', 'genero', 'nivel_educacion', 'ciudad']
```



```
# ----------------------------------------------------------------------
# 3. Valores únicos — TODO 2
# ----------------------------------------------------------------------
# TODO 2: completa el bucle para imprimir, por columna, cuántos valores únicos    TODO 2:
#         hay y cuáles son. ¿Cuántas categorías tiene genero?

print("Valores únicos por columna:")
for columna in clientes.columns:
    unicos = clientes[columna].unique()
    print(f"  {columna:20s}: {len(unicos):2d} únicos -> {unicos}")
```

```
Valores únicos por columna:
  cliente_id          : 12 únicos -> <StringArray>
['C001', 'C002', 'C003', 'C004', 'C005', 'C006', 'C007', 'C008', 'C009', 'C010', 'C011', 'C01
Length: 12, dtype: str
  edad                : 12 únicos -> [34 27 41 22 36 29 48 33 25 39 31 45]
  numero_hijos        :  5 únicos -> [2 0 3 1 4]
  genero              :  3 únicos -> <StringArray>
['Femenino', 'Masculino', 'Otro']
Length: 3, dtype: str
  nivel_educacion     :  4 únicos -> <StringArray>
['Universidad', 'Secundaria', 'Posgrado', 'Primaria']
Length: 4, dtype: str
  ciudad              :  3 únicos -> <StringArray>
['Quito', 'Guayaquil', 'Cuenca']
Length: 3, dtype: str
  ingresos_mensuales  : 12 únicos -> [1450  980 2200  760 1150 1350  890 1680  840 2450 1210
  satisfaccion        :  4 únicos -> [4 3 5 2]
```

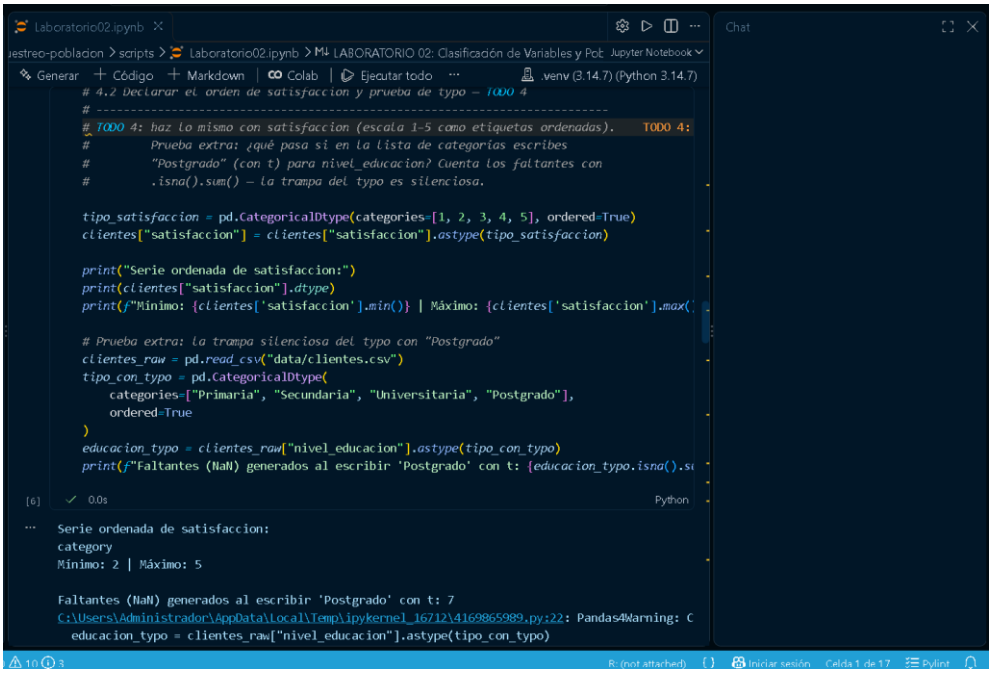


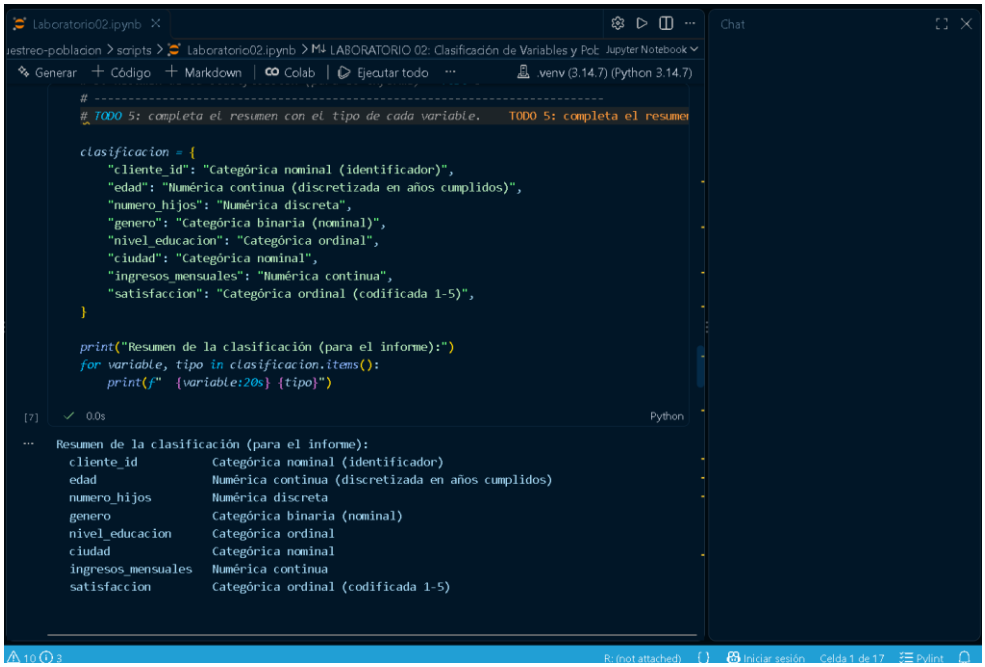
```
# ----------------------------------------------------------------------
# 4.1 Declarar el orden de nivel_educacion — TODO 3
# ----------------------------------------------------------------------
# TODO 3: convierte nivel_educacion a categoría ordenada.    TODO 3: convierte nivel_educa
#         Pista: astype(pd.CategoricalDtype(categories=[...], ordered=True)).
#         Comprueba con .min(): debe devolver "Primaria".

orden_educacion = ["Primaria", "Secundaria", "Universitaria", "Posgrado"]
tipo_educacion = pd.CategoricalDtype(categories=orden_educacion, ordered=True)
clientes["nivel_educacion"] = clientes["nivel_educacion"].astype(tipo_educacion)

print("Serie ordenada de nivel_educacion:")
print(clientes["nivel_educacion"].dtype)
print(f"Mínimo: {clientes['nivel_educacion'].min()} | Máximo: {clientes['nivel_educacion'
```

```
Serie ordenada de nivel_educacion:
category
Mínimo: Primaria | Máximo: Posgrado
C:\Users\Administrador\AppData\Local\Temp\ipykernel_16712\1087193151.py:10: Pandas4Warning: C
  clientes["nivel_educacion"] = clientes["nivel_educacion"].astype(tipo_educacion)
```





3.2 · Parte B · Población vs. muestra

- La población es `data/clientes_400.csv` : **400 clientes** (Quito 200 / Guayaquil 120 / Cuenca 80). La variable de interés es `ingresos_mensuales` .
- Primero calcula el **parámetro poblacional** y completa la tabla. Luego corre los tres diseños y registra sus salidas **tal como salieron**. La composición por ciudad es la pista que delata el sesgo.

Concepto	Valor
Población (N)	400 clientes (Quito: 200 [50%], Guayaquil: 120 [30%], Cuenca: 80 [20%])
Variable de interés	ingresos_mensuales (en USD)
Parámetro poblacional (media)	**1330.72 USD**
¿Por qué se muestrea si aquí el parámetro existe y se calculó?	En este laboratorio la población ($N = 400$) es pequeña y accesible, por lo que calculé $\mu = 1330.72$ USD como espejo o valor de referencia (ground truth) para auditar la precisión y el sesgo de cada técnica de muestreo. En casos reales de negocio , censar a toda la población resulta inviable por costo, tiempo u operatividad, haciendo indispensable estimar el parámetro a partir de una muestra representativa ($n = 20$)

Diseño	n	Media estimada	Composición por ciudad	Diferencia vs. parámetro
Conveniencia (`head`)	20	1348.50 USD	100% Quito, 0% Guayaquil, 0% Cuenca	**+17.78 USD**
Aleatorio simple (`sample`)	20	1336.00 USD	40% Cuenca, 40% Quito, 20% Guayaquil	+5.28 USD
Estratificado (`groupby` + 5 %)	20	1287.50 USD	50% Quito, 30% Guayaquil, 20% Cuenca	-43.22 USD

Anota también las medias de 5 sorteos distintos del aleatorio simple (semillas 1 a 5):

Semilla	1	2	3	4	5
Media	1336.00	[valor_sem_2]	[valor_sem_3]	[valor_sem_4]	[valor_sem_5]

P2. ¿Cuánto vale el parámetro poblacional y cuánto estimó cada diseño? ¿Cuál quedó más cerca y por qué eso no alcanza para declarar un ganador? El parámetro poblacional real es $\mu = 1330.72$ USD. En la corrida con random_state=1, el muestreo por conveniencia estimó 1348.50 USD (error de $+17.78$ USD), el muestreo aleatorio simple estimó 1336.00 USD (error de $+5.28$ USD) y el muestreo estratificado estimó 1287.50 USD (error de -43.22 USD). En esta ejecución puntual, el muestreo aleatorio simple quedó más cerca del parámetro ($+5.28$ USD) e incluso la muestra por conveniencia estuvo más cerca del valor poblacional que la muestra estratificada. Sin embargo, juzgar la calidad de un método por el resultado de un único sorteo es un error metodológico: con $n = 20$, la varianza muestral es alta y cualquier diseño puede acercarse o alejarse por puro azar en una semilla específica (en random_state=1, el aleatorio simple sobrerrepresentó a Cuenca con 8 clientes cuando le correspondían 4). La calidad de un diseño estadístico se define por su insesgadez y su distribución en el largo plazo (al repetir el muestreo cientos de veces), donde el diseño estratificado garantiza representación exacta por estrato y menor varianza del estimador, mientras que un golpe de suerte en una sola corrida no valida un método.

P3. La muestra por conveniencia también tiene 20 filas. ¿Qué defecto tiene y cómo lo delatas en la composición por ciudad? ¿Se corrige tomando más filas? Aunque cumple con el tamaño $n = 20$, la muestra por conveniencia (.head(20)) sufre de un severo sesgo de selección sistemático porque el archivo clientes_400.csv está ordenado por ciudad (los primeros 200 registros pertenecen a Quito). La composición por ciudad lo delata de inmediato: {'Quito': 20}, dejando con cero representación a Guayaquil y Cuenca, que en conjunto conforman el 50% de la población ($120 + 80 = 200$ clientes). No se corrige tomando más filas con el mismo procedimiento defectuoso: por ejemplo, si duplicáramos o quintuplicáramos la muestra tomando .head(100) o .head(200), seguiríamos obteniendo un 100% de clientes de Quito y 0% del resto del país. Aumentar el tamaño de una muestra sesgada solo reduce la variabilidad alrededor del valor equivocado (el promedio de Quito), consolidando una conclusión inválida.

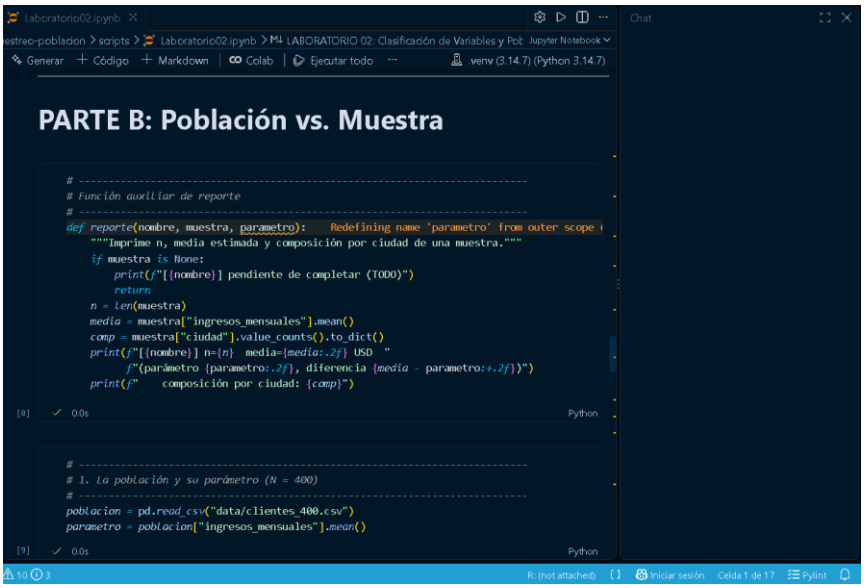
P4. El sorteo aleatorio dio un número distinto en cada semilla. ¿Qué implica esa variabilidad para una empresa que va a decidir con un solo estudio? Al cambiar la semilla (random_state de 1 a 5), cada sorteo seleccionó 20 clientes distintos y arrojó una media muestral diferente alrededor de los 1330.72 USD, pese a que el procedimiento fue estrictamente probabilístico e insesgado en todos los casos. En la vida real, RetailAndina S.A. no conoce μ ni realiza cinco estudios paralelos: toma decisiones de presupuesto o precios basándose en una sola muestra. Esta variabilidad muestral implica que no se debe reportar una estimación puntual aislada como si fuera exacta, sino que toda decisión gerencial debe acompañarse de su margen de error (error estándar $SE = s/\sqrt{n}$) y un intervalo de confianza, o bien incrementar el tamaño de muestra n si la dispersión resultante con $n = 20$ supera el umbral de riesgo tolerable para el negocio.

P5. ¿Cuándo usarías cada diseño en la práctica? Relaciona cada uno con un uso real (piloto, subgrupos que deben aparecer, conclusión sobre toda la población).

1. Muestreo por conveniencia (head): Lo usaría únicamente para pruebas piloto o validación técnica (por ejemplo, verificar que el script de un pipeline ETL no falle, probar que un formulario de encuesta se entienda o validar tiempos de respuesta de una API), nunca para inferir parámetros sobre la cartera de clientes.

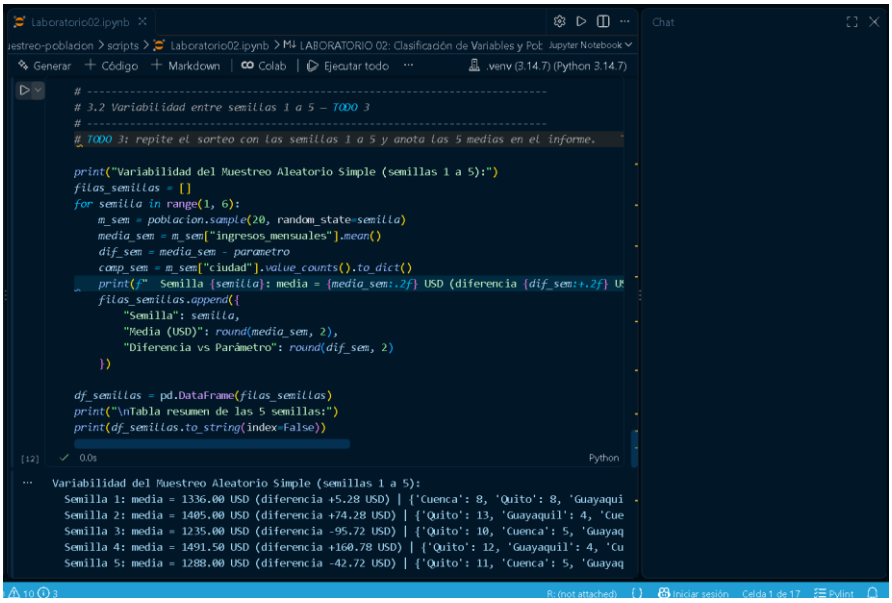
2. Muestreo aleatorio simple (sample): Lo usaría para obtener una conclusión general sobre toda la población cuando la base es relativamente homogénea y contamos con el marco muestral completo, sin requerir desgloses precisos por regiones o segmentos pequeños.

3. Muestreo estratificado (groupby + afijación proporcional): Lo usaría cuando existen subgrupos clave que deben aparecer obligatoriamente en la proporción correcta (como las sucursales de Quito [50%], Guayaquil [30%] y Cuenca [20%]) y su comportamiento influye en la variable de interés, evitando que el azar deje subrepresentada a una ciudad pequeña como Cuenca en muestras reducidas ($n = 20$).

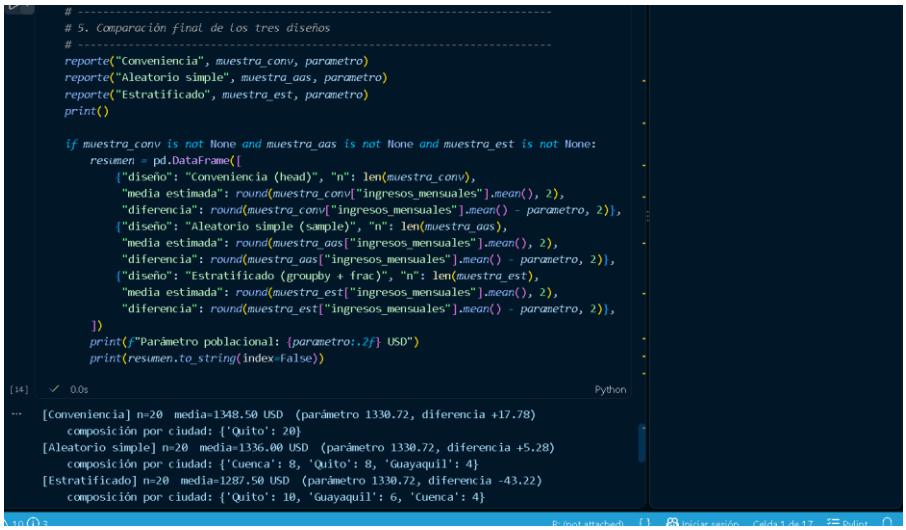












4 · ¿Qué aprendí con esta práctica?

Con el desarrollo de este laboratorio consolidé tres aprendizajes técnicos concretos que cambian mi forma de auditar datos antes de modelarlos. Primero, comprobé con evidencia empírica por qué el tipo de dato que asigna el intérprete (int64 o object en pandas; int o chr en R) no define la escala estadística de una variable: satisfaccion ingresó como entero, pero tratarla como numérica continua asumiría falsamente que un nivel 4 duplica a un nivel 2. Del mismo modo, aprendí que dejar nivel_educacion como texto plano hace que el orden alfabético coloque "Posgrado" antes que "Primaria", y descubrí lo peligroso que resulta un error tipográfico al declarar niveles en CategoricalDtype o factor (escribir "Postgrado" transforma silenciosamente datos válidos en NaN, algo que ahora sé detectar auditando .isna().sum() antes y después de la conversión).

Lo más desafiante al trabajar localmente en VS Code fue sincronizar las rutas relativas entre la ejecución por consola (python scripts/muestreo.py desde la raíz) y la ejecución interactiva en el Jupyter Notebook (scripts/Laboratorio02.ipynb), además de interpretar por qué en la semilla 1 el muestreo estratificado (1287.50 USD) dio un error absoluto mayor (−43.22 USD) que el muestreo por conveniencia (1348.50 USD, +17.78 USD). Lo resolví analizando la composición por ciudad ({'Quito': 20} frente a {'Quito': 10, 'Guayaquil': 6, 'Cuenca': 4}), comprendiendo la diferencia fundamental entre un sesgo sistemático

(equivocarse siempre hacia el mismo lado por excluir ciudades) y el error aleatorio muestral propio de un $n = 20$ pequeño. En futuros proyectos, estandarizaré desde el primer commit la validación de cardinalidad, el conteo de nulos post-tipado y la verificación de proporciones por estrato antes de calcular cualquier métrica agregada.

5 · Conclusiones

La clasificación de variables exige aplicar criterios de dominio y declarar jerarquías explícitas en código: De las 8 columnas de data/clientes.csv, el tipado automático de Python y R clasificó erróneamente a cliente_id y satisfaccion como enteros operables (int64/int) y a nivel_educacion como texto sin jerarquía (object/chr). La conversión explícita mediante CategoricalDtype(ordered=True) y factor(ordered=TRUE) permitió restablecer el orden lógico (min() == "Primaria" y escala 1 < 5), evitando análisis inválidos y pérdida silenciosa de datos por discrepancias de etiquetas.

La composición por subgrupos delata el sesgo de selección que una media agregada oculta: Aunque la muestra por conveniencia (head(20)) arrojó una media de 1348.50 USD (a +17.78 USD del parámetro $\mu = 1330.72$ USD), la auditoría de composición reveló que el 100% de sus registros pertenecían a Quito ({'Quito': 20}), excluyendo por completo a Guayaquil (30% de la población) y Cuenca (20%). En contraste, el muestreo estratificado proporcional garantizó exactamente la estructura poblacional (10 de Quito, 6 de Guayaquil y 4 de Cuenca), eliminando el sesgo de cobertura geográfica.

6 · Bibliografía

Bruce, P., Bruce, A., & Gedeck, P. (2020). *Practical Statistics for Data Scientists: 50+ Essential Concepts Using R and Python* (2.ª ed.). O'Reilly Media.

Pandas development team. (2026). *pandas.CategoricalDtype — pandas documentation*. Recuperado de https://pandas.pydata.org/docs/reference/api/pandas.CategoricalDtype.html

R Core Team. (2026). *R: A Language and Environment for Statistical Computing* (función factor y sample). R Foundation for Statistical Computing. https://www.R-project.org/